

Command-line tool to classify patients as exposed or naïve to antiretrovirals based on the HIV-1 *pol* gene using Hidden Markov Models

Manuel Almeida¹, Maria Leonor Frazão¹, Cruz S. Sebastião^{1,2,3}, André Santos¹, Mafalda Miranda¹, Inês Alves¹, Marta Pingarilho¹, Marta Giovanetti⁴, Ana B. Abecasis¹, **Victor Pimentel¹**

Author's details

¹ Global Health and Tropical Medicine (GHTM), Associate Laboratory in Translation and Innovation Towards Global Health (LA-REAL), Instituto de Higiene e Medicina Tropical (IHMT), Universidade NOVA de Lisboa (UNL), Rua da Junqueira 100, 1349-008 Lisboa, Portugal

² Centro Nacional de Investigação Científica (CNIC), Luanda, Angola

³ Instituto Nacional de Investigação em Saúde (INIS), Luanda, Angola

⁴ Department of Science and Technology for Humans and the Environment, University of Campus Bio-Medico di Roma, Rome, Italy

Abstract

Introduction: High-potency antiretroviral therapy has increased life expectancy and reduced HIV transmission, but its expansion has revealed resistance mutations. In Portugal, over 50% of new HIV diagnoses involve migrants, many of whom already know their HIV status and have had prior therapy but report as new cases due to stigma. This results in "false" transmitted resistance, complicating first-line treatment guidelines. This study aimed to develop a machine learning algorithm to classify patients as treated or naïve based on *pol* gene sequence analysis.

Methods: 25,275 sequences (subtypes B, CRF02_AG) from the Los Alamos database, covering Protease and Reverse Transcriptase regions, were aligned using MAFFT and manually edited to obtain an 1100 nt fragment (positions 2253-3363 relative to HXB2). Sequences were translated, and major resistance codons (WHO-2009 list) were removed. Hidden Markov Models (HMMs) using Match (M), Insertion (I), and Deletion (D) states were trained, with transition and emission probabilities estimated from labelled sequences. The Forward algorithm computed a log odds ratio, comparing the sequence's probability under the motif model to a null model, providing a score for motif likelihood. Model performance was assessed using accuracy, precision, recall, and F1 score.

Results: The HMM classifier was efficient in identifying naïve vs. treated individuals, with average (from 35 iterations) performance metrics for subtype B: accuracy (67.77%), precision (71.47%), recall (67.77%), and F1 score (66.32%). For subtype CRF02_AG: accuracy (65.97%), precision (74.39%), recall (65.97%), and F1 score (62.75%).

Discussion: Bayesian classification of therapeutic status offers a simple, resource-efficient approach to improve HIV management in populations with uncertain treatment histories, addressing challenges posed by stigma and resistance mutations.

KEYWORDS: HIV-1; Treatment; Drug resistance; Machine learning

Contents

Introduction	3
Context	3
HIV in Portugal.....	3
Significance	3
Introduction to Hidden Markov Models	4
Hidden Markov Models for Biological Sequences.....	5
Methodology.....	6
Exploring and Cleaning Data.....	7
Model Training	8
Model Testing	8
Building a command-line program.....	9
Use of AI-Assisted Development.....	10
Results	10
Model Performance.....	10
Command-line program	10
Discussion	16
Bibliography	16

Introduction

Context

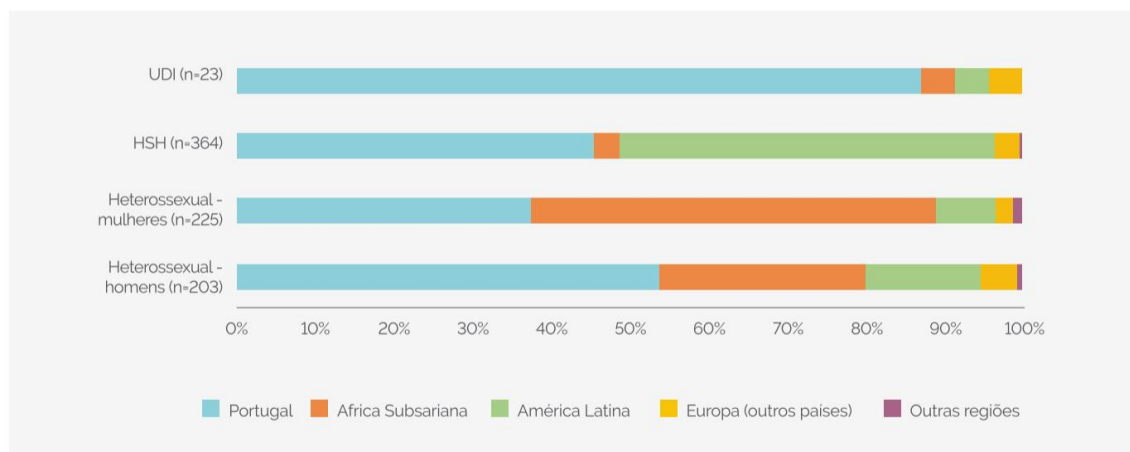
This project was conducted within the curricular unit *Project in Multi-Omics* of the Master's in Computational Biology and Bioinformatics at NOVA School of Science and Technology, in collaboration with the THOP research group at IHMT under the supervision of Dr. Victor Pimentel. This curricular unit aimed to immerse students in the ongoing research workflow for two weeks of hands-on experience, followed by a scientific writing phase.

The Institute of Hygiene and Tropical Medicine (IHMT), a unit of NOVA University Lisbon under the Ministry of Education and Science, focuses on advancing scientific knowledge in health issues related to tropical and intertropical regions.

IHMT's research is integrated within the Global Health and Tropical Medicine (GHTM) center, which hosts multiple research groups, including THOP, specializing in molecular epidemiology, drug resistance mechanisms, and the development of novel diagnostics and treatments for tuberculosis, HIV, and opportunistic pathogens.

HIV in Portugal

In 2023, 924 new HIV cases were recorded in Portugal. Among the 863 cases with available birth country data, 46.9% (405) involved individuals born in Portugal, while 53.1% (458) were foreign-born, constituting the majority of new diagnoses. Of these, 49.8% originated from Latin America, and 42.4% from Sub-Saharan Africa (Portugal. Ministério da Saúde. Direção-Geral da Saúde/Instituto Nacional de Saúde Doutor Ricardo Jorge, 2024).



Legenda: HSH –homens que têm sexo com homens; UDI – utilizadores de drogas injetadas

Figure 1 - Novos casos de infeção por VIH (≥ 15 anos) com diagnóstico em 2023: distribuição (%) da origem geográfica dos indivíduos por modo de transmissão.

Significance

The expansion of high-potency antiretroviral therapy (ART) has prolonged life expectancy and reduced HIV transmission. However, incorrect prescription and treatment interruptions have contributed to drug resistance mutations (DRMs) within the HIV genome.

A major challenge arises from individuals with prior ART exposure but no accessible medical records. Many opt to conceal their treatment history due to social stigma, complicating clinicians' ability to prescribe appropriate therapy. In the absence of treatment history, first-line prescriptions often rely on broad-spectrum regimens until further diagnostic insights are available (Pimentel et al., 2024).

A cost-effective and rapid tool capable of distinguishing ART-experienced from ART-naïve patients would be invaluable for optimizing treatment strategies in these cases.

Introduction to Hidden Markov Models

Hidden Markov Models (HMMs) provide a probabilistic framework for modeling sequential data, where observations are generated by a hidden state system. The key assumption is that the hidden states follow the Markov property, meaning each state depends only on the previous one.

In a first-order HMM, the state at time t (denoted Z_t) is conditioned solely on the state at $t-1$ (Z_{t-1}). Higher-order models allow dependency on multiple prior states. A graphical representation of a first-order HMM is illustrated in Figure 2 (Degirmenci, 2014).

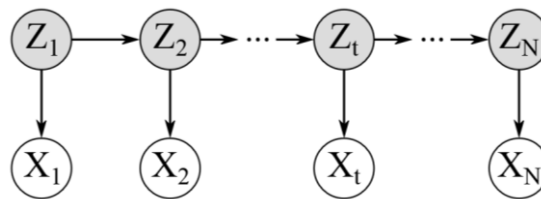


Figure 2 - A Bayesian network representing a first-order HMM. The hidden states are shaded in grey (Degirmenci, 2014).

While the term "time" is often used to describe sequential observations, it does not necessarily indicate temporal progression; rather, it can represent any ordered sequence, such as spatial or structural arrangements (Degirmenci, 2014).

The joint probability distribution of a hidden state sequence $Z_1:Z_N$ and corresponding observations $X_1:X_N$ in a first-order HMM is governed by:

$$P(Z_{1:N}, X_{1:N}) = P(Z_1)P(X_1|Z_1) \prod_{t=2}^N P(Z_t|Z_{t-1})P(X_t|Z_t)$$

where:

- $P(Z_1)$ represents the initial state probability,
- $P(Z_t|Z_{t-1})$ denotes transition probabilities between hidden states,
- $P(X_t|Z_t)$ corresponds to emission probabilities of observations given the hidden states.

Hidden Markov Models for Biological Sequences

There are several viable approaches to applying Hidden Markov Models (HMMs) to biological sequences. This study builds on the work of Professor Dannie Durand (Carnegie Mellon University) as outlined in *Computational Biology* (2019), chapter *Hidden Markov Models*. Consider the following multiple-sequence alignment:

```

VG--H
V---N
VE--D
IAADN

```

The length of a Profile HMM should correspond to the typical length of the motif being modeled. A common heuristic is to use the average length of the training sequences. In this example, the sequences have lengths of 3, 2, 3, and 5 (prior to the introduction of gaps), yielding an average of 3.25. This suggests that a Profile HMM with a length of 3 is appropriate for capturing this pattern. The resulting HMM consists of a silent Start state (M_0), Match states (M_1, M_2, M_3), Insertion states (I_0, I_1, I_2, I_3), Deletion states (D_1, D_2, D_3), and a silent End state (M_4). The Match and Insertion states emit the 20 standard amino acids (for protein motifs) or the four nucleotides (for DNA and RNA motifs), while Deletion states emit gap symbols (“-”). The emission probabilities for Insertions and Deletions follow the distribution:

$$e_{D_j}(\sigma) = 0, \quad e_{D_j}(-) = 1$$

$$e_{I_j}(\sigma) = p(\sigma), \quad \forall I_j$$

where $p(\sigma)$ represents the background probability of residue σ . To estimate the Match state parameters, labels are assigned to the data based on the multiple-sequence alignment.

Columns in the alignment containing gaps in fewer than half of the sequences are designated as Match states, whereas those with gaps in more than half of the sequences correspond to Insertion states:

V	G	-	-	H
V	-	-	-	N
V	E	-	-	D
I	A	A	D	N
M_1	M_2	I_2	I_2	M_3

This process yields the following labelled sequences:

V	G	H		
M_1	M_2	M_3		
V	-	H		
M_1	D_2	M_3		
V	E	D		
M_1	M_2	M_3		
I	A	A	D	N
M_1	M_2	I_2	I_2	M_3

When a gap (“-”) appears in a Match column (Mi), it is labeled as a Deletion (Di). For instance, the first gap in the second sequence is labeled as Di. These labeled sequences allow for the estimation of emission and transition probabilities. For example, applying a pseudocount of 1, the emission probabilities are calculated as:

$$e_{M_1}(V) = \frac{3 + 1}{4 + 20}.$$

Similarly, the probability of a transition from M2 to I2 is

$$a_{M_2I_2} = \frac{1 + 1}{(2 + 1) + (1 + 1) + (0 + 1)}.$$

The three summations in the denominator account for all possible transitions out of state M2. The first term in each sum corresponds to the number of observed transitions in the training data, while the second term is the applied pseudocount. In this dataset, there are two transitions from M2 to M3, one transition from M2 to I2, and none from M2 to D3. The remaining emission and transition probabilities are computed following the same approach. The model always initiates from M0, ensuring that $\pi_{Mj}=0$ for all $j>0$.

Once the Profile HMM has been constructed, the classification of an unknown sequence OOO can be determined by computing a log-odds ratio, which compares the sequence’s probability under the Profile HMM (HA) against a background model (H0):

$$\log \frac{P(O|H_A)}{P(O|H_0)},$$

The Forward algorithm is used to compute these probabilities, providing a statistical measure of how likely a sequence is to contain the motif. Typically, HA represents the Profile HMM trained on the motif of interest, while H0 serves as a background model, distinguishing relevant sequences from random noise.

Methodology

The dataset utilized in this study comprised 25,275 sequences (subtypes B and CRF02_AG) obtained from the Los Alamos database, encompassing the Protease and Reverse Transcriptase regions. These sequences were translated and aligned using MAFFT, followed by manual refinement to extract an 1100-nucleotide fragment spanning positions 2253–3363 relative to the HIV-1 reference genome (HXB2).

The amino acid sequences were formatted into a CSV file, structured as a table containing the following information: sequence identification (Header), subtype classification (AG_02 or B), treatment status (ART-treated or naïve), and the corresponding amino acid sequences with gap indicators. The dataset included a total of 25,275 sequences, with the first sequence corresponding to the HIV-1 reference genome (HXB2).

To facilitate data analysis, the sequences were initially converted into a data frame (Figure 3) using the Pandas library in *Python*. Subsequent computational procedures were implemented through *Python* scripts.

	Header	Subtype	Treatment	1	2	3	4	...	428	429	430	431	432	433	434
0	HXB2_pr_rt	B	Naive	P	Q	V	T	...	I	Q	K	Q	G	Q	G
1	KR867938	02_AG	Treated	P	Q	I	T	...	I	Q	K	Q	G	Q	D
2	KR867957	02_AG	Treated	P	Q	I	T	...	?	Q	K	Q	G	Q	D
3	KR867960	02_AG	Treated	P	Q	I	T	...	I	Q	K	Q	G	Q	D
4	KR868147	02_AG	Treated	P	Q	I	T	...	I	Q	K	Q	G	Q	D
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
25270	AB866154	B	Naive	P	Q	I	T	...	-	-	-	-	-	-	-
25271	AB866155	B	Naive	P	Q	I	T	...	-	-	-	-	-	-	-
25272	AB866157	B	Naive	P	Q	I	T	...	-	-	-	-	-	-	-
25273	AB866156	B	Naive	P	Q	I	T	...	-	-	-	-	-	-	-
25274	AB866158	B	Naive	P	Q	I	T	...	-	-	-	-	-	-	-

[25275 rows x 437 columns]

Figure 3 – *Print* of the dataframe containing the raw data.

Exploring and Cleaning Data

Handling Data and Preprocessing for HMM

Unlike most machine learning algorithms, Hidden Markov Models (HMMs) operate directly on sequences of symbols, eliminating the need for vectorization of the amino acid sequences. Likewise, there's no benefit in removing gaps within sequences, because they provide valuable structural information for the HMM profiles, and therefore, should not be removed.

Unbalanced Dataset

The dataset contained 15,568 ART-naïve sequences and 9,677 ART-experienced sequences, resulting in an imbalance that could affect model training. To mitigate this issue, undersampling of naïve sequences was employed to create a balanced dataset.

Background Probabilities

HMMs require background probabilities (BP) for amino acid symbols. To facilitate access across scripts, a background probabilities dictionary (BPD) was created and stored in a separate *Python* file, where amino acids serve as dictionary keys and their respective probabilities as values.

Initially, BP values were based on their general frequency across all life forms. However, a more biologically relevant approach was later adopted by computing BPs specific to HIV-1, using the HXB2 reference sequence. An auxiliary script counted symbol occurrences within HXB2, divided by sequence length to derive frequency estimates, and assigned zero probability to gap symbols.

Although an alternative approach involved computing BP values across all dataset sequences, this risked overfitting the BP model to the training data. Using HXB2 as a reference was deemed a more reliable choice due to its widespread use in HIV-related studies.

Handling Ambiguity Codes

The dataset contained sequences with ambiguity codes ("?"), which, according to the expanded IUPAC amino acid alphabet, can represent any of the 20 amino acids. This posed a challenge for HMM implementation, as no probability was assigned to "?" in the BPD.

A theoretical approach was considered: assigning the probability of any amino acid occurrence ($1 - P("-")$). However, this resulted in extreme probability values (1 for M and I states, 0 for D states), disproportionately influencing forward probability calculations.

Another strategy treated "?" as a distinct amino acid symbol, assigning its BP as its relative frequency in the dataset. However, as HXB2 contains no ambiguity codes, this method would incorrectly assign zero probability to "?". Additionally, deriving BP values from the entire dataset risked overfitting.

The final adopted approach involved replacing all "?" occurrences with the corresponding amino acid from the HXB2 sequence, ensuring biologically meaningful substitution without skewing probability calculations.

Removing Drug Resistance Mutation (DRM) Codons

To prevent bias in classification, Dr. Victor Pimentel recommended the removal of major drug-resistance mutation (DRM) codons from the dataset. The rationale was that sequences containing known DRM sites could be erroneously classified as "treated," even when resistance mutations were transmitted rather than acquired.

Initial model performance tests were conducted with DRM codons removed (WHO-2009 list). However, in the final command-line program, the option to include or exclude DRM codons was left to user discretion.

Data Splitting Strategy

For model training, sequences were divided into distinct classification groups, followed by a split into training and test datasets. Dr. Victor Pimentel strongly recommended splitting sequences by HIV subtype before stratifying by treatment status, as this improved classification performance. The default test fraction was set to 20%, but this value remains user-configurable in the command-line program.

Model Training

In the context of Hidden Markov Models (HMMs), training involves constructing the Profile HMM, which requires estimating transition and emission probabilities. This computation was straightforward, and the resulting implementation proved efficient, with each profile HMM being generated within approximately 1–2 seconds, meeting performance requirements.

Model Testing

The evaluation of Hidden Markov Models (HMMs) involved scoring each sequence in the testing set against all profile HMMs and assigning classifications based on the highest-

scoring profile. Performance was assessed using accuracy, precision, recall, and F1 score by verifying whether classifications were correct.

An initial issue arose due to the underflow limit of *Python* ($\sim 2.2251e-308$), causing probabilities computed by the Forward algorithm to be rounded to zero. This was a critical problem, as zero probabilities in HMMs differ significantly from near-zero probabilities, leading to incorrect likelihood calculations.

To mitigate underflow, all probability computations were converted to log space using base-10 logarithms, allowing calculations to remain within a numerically stable range. This approach was also applied to the background probability model.

While using log space prevented underflow issues, the Forward algorithm initially exhibited inefficiencies, primarily due to the presence of multiple nested loops, leading to execution times of 30 seconds to one minute per sequence. To enhance performance, the following optimizations were implemented:

- Replacing nested loops with recursively defined lists, reducing redundant computations.
- Scaling probability values to minimize unnecessary logarithmic operations.

These modifications significantly improved runtime, reducing computation time to a few seconds per sequence, aligning with the performance requirements of the study.

To ensure reliable and generalizable results, multiple iterations of training and testing were conducted. Each iteration involved random selection of sequences for training and testing, reducing potential biases and improving the statistical robustness of the performance metrics.

Building a command-line program

To integrate the previously developed code into a functional command-line program, the *Argparse* package was utilized, as it is the standard for *Python*-based CLI applications. This implementation ensures that the program remains scalable and adaptable for future modifications and broader user accessibility.

To maintain compatibility across different operating systems, the *OS* and *Pathlib* packages were employed for directory management, enabling seamless navigation and file handling.

User accessibility was a key consideration, particularly for individuals with minimal command-line experience. Therefore, the user interface (UI) was designed to be highly interactive, providing detailed error messages that indicate potential causes rather than simply terminating execution.

For version control of dependencies and *Python* itself, the installation process was standardized through the creation of a virtual environment, ensuring that the correct package versions are installed via a *requirements.txt* file. This approach enhances reproducibility and system compatibility across different setups.

Use of AI-Assisted Development

To accelerate development, AI-based tools such as *ChatGPT* and *DeepSeek* were utilized for generating auxiliary functions, including directory navigation, file processing, and UI enhancements (e.g., animations and progress bars). The time saved through automation enabled greater focus on refining HMM implementation and model optimization

Results

Model Performance

The models trained were respective to the subtypes present in the data (AG_02 and B). To balance the data set we used under sampling. The DRM codons from WHO-2009 list were removed. The results mentioned in Table 1 correspond to the performance metrics of both models, with 35 iterations per model.

Table 1 - Model performance metrics

Subtype	B			CRF02_AG		
Metrics (%)	Precision	Recall	F1-score	Precision	Recall	F1-score
Treated	62,16	88,66	73,09	88,72	36,59	51,81
Naïve	80,24	46,04	58,51	60,06	95,35	73,7
Mean	71,47	67,77	66,32	74,39	65,97	62,75

Both models resulted in a similar accuracy, subtype B with 67,77% accuracy, and subtype CRF01_AG with 65,97% accuracy.

Command-line program

```
C:\Users\Asus\Documents\GitHub\HIV_HiddenMarkovModels_naive-treated_Prediction\code>python hiv_hmm_pred.py -h
=====
HIV-1 ART-naive/treated prediction with HMM - v1.0
=====
USAGE:
python hiv_hmm_pred.py [OPTIONS]

DESCRIPTION:
This program takes HIV-1 "pol" gene sequences (in FASTA
format) and predicts if they belong to a patient who has
experienced antiretroviral treatment (ART-treated) or not
(ART-naïve), using Hidden Markov Models.

OPTIONS:
-h, --help
    Display this help message and exit.

-c, <IN_FILE>, --classify <IN_FILE>
    Classify sequences in the provided FASTA file.

    (Sub-option)
    -o, <OUT_FILE>, --output <OUT_FILE>
        Specify an output CSV file path for classification.
        (Default: output_data/output.csv)

-t, --test
    Test performance of HMM algorithm.

ARGUMENTS:
  IN_FILE  Fasta file with sequences to classify (must be
           located on the "input_data" folder)
  OUT_FILE Specific CSV file for the output.

=====
Developed by : Manuel Almeida
Email address: mpa.almeida@campus.fct.unl.pt
=====
Available in: https://github.com/ml3paiva/IHMT-HIV-naive-experienced-prediction
```

Figure 4 – Help message

All the files and instructions (README.md file) necessary to install and run the program are available at:

➔ https://github.com/m13paiva/HIV_HiddenMarkovModels_naive-treated_Prediction

The help message in **Erro! A origem da referência não foi encontrada.** shown by the help command (“-h”), shows the available commands. There are two main commands– the *classify* command, and the *test* command.

Classify command

This command is responsible for the aim of this project, classifying patients as treated or naïve to antiretrovirals, by only providing their HIV virus’ *pol* gene sequences.

As mentioned in the help message (**Erro! A origem da referência não foi encontrada.**), requires a *IN_FILE* argument, which is the *FASTA* file containing the amino acid sequences that the user pretends to classify, and this file should be located on the directory “input_data”. Optionally, the user could also specify a name for the CSV output file with the “-o” sub-option (default is “output.csv”).

If no file is provided the following message is presented:

```
C:\Users\Asus\Documents\GitHub\HIV_HiddenMarkovModels_naive-treated_Prediction\code>python hiv_hmm_pred.py -c
usage: hiv_hmm_pred.py [-h] [-c IN_FILE] [-o OUT_FILE] [-t]
hiv_hmm_pred.py: error: argument -c/--classify: expected one argument
```

If the provided file does not exist in the “input_data” directory:

```
C:\Users\Asus\Documents\GitHub\HIV_HiddenMarkovModels_naive-treated_Prediction\code>python hiv_hmm_pred.py -c file_that_does_not_exist.fasta
The file 'C:\Users\Asus\Documents\GitHub\HIV_HiddenMarkovModels_naive-treated_Prediction\input_data\file_that_does_not_exist.fasta' does not exist!
Please indicate a valid FASTA file.
```

If the provided file isn’t in the *FASTA* format:

```
C:\Users\Asus\Documents\GitHub\HIV_HiddenMarkovModels_naive-treated_Prediction\code>python hiv_hmm_pred.py -c not_a_fasta_file.txt
ERROR: Wrong file format!
Please indicate a FASTA file (must end with .fasta).
```

When a correct file is provided the program initializes the *classify* function:

```
C:\Users\Asus\Documents\GitHub\HIV_HiddenMarkovModels_naive-treated_Prediction\code>python hiv_hmm_pred.py -c test.fasta

=====
Hello Dr. Asus! Ready to find who's naive and who's treated?
=====

Please write the HIV subtype of the sample(s).
("A", "B", "02_AG" or "unknown")
```

Firstly, the program asks what is the subtype of the HIV sequences provided. When the user doesn’t know yet what is subtype of his samples, he’s asked if he’d like to predict that as well, or just stick with the treated/naïve classification (the inputs are not case sensitive):

```

C:\Users\Asus\Documents\GitHub\HIV_HiddenMarkovModels_naive-treated_Prediction\code>python hiv_hmm_pred.py -c test.fasta
■
=====
Hello Dr. Asus! Ready to find who's naïve and who's treated?
=====

Please write the HIV subtype of the sample(s).
("A", "B", "02_AG" or "unknown")

>>>> UnKnOwN

I see that you don't know the subtype of your sample(s).
Would you like to predict the subtype as well?
(y/n)

>>>> Y

=====

Do you want the model to discard the major DRM codons?
(y/n)

>>>>

```

Next, the program asks if the user wants to classify without using the major DRM codons. If the user decides to discard these codons, then it asks if the user wants to specify a different list of DRM codons:

```

Do you want the model to discard the major DRM codons?
(y/n)

>>>> y

The major DRM codons considered by the model are the following:
» PI codons: [30, 32, 33, 46, 47, 48, 50, 54, 76, 82, 84, 88, 90]
» RT codons: [184, 65, 70, 74, 115, 41, 67, 70, 210, 215, 219, 100, 101, 103, 106, 138, 181, 188, 190, 230]

Do you want to specify a different list of DRM codons?
(y/n)

>>>> y

Specify Protease Inhibitor (PI) codons.
List of codon positions must be given in the format 'pos1, pos2, ..., pos N'.

PI codons >>>> 1,4,7,78

Specify Reverse Transcriptase (RT) codons.
List of codon positions must be given in the format 'pos1, pos2, ..., pos N'.

RT codons >>>> 2,8,9

=====

Would you like the program to remember your choices?
(y/n)

>>>> y

Your choices have been registered!
The next time the program is ran, you'll be asked if you want
to execute your previous choices.

=====

Preparing profile data . . ■

```

In the end, the program asks if the user wants to remember every choice he's made so far (except HIV subtype), so the next time the program is ran it will ask if the user wants to use the previous choices (Figure 4). The choice remembering function is very important because, when repeated multiple times (analysing multiple files), this can be a tedious task.

```

C:\Users\Asus\Documents\GitHub\HIV_HiddenMarkovModels_naive-treated_Prediction\code>python hiv_hmm_pred.py -c test.fasta
=====
Hello Dr. Asus! Ready to find who's naive and who's treated?
=====

Do you want to use the choices previously saved?
(y/n)

>>>> y

=====

Please write the HIV subtype of the sample(s).
("A", "B", "02_AG" or "unknown")

>>>> B

=====

Preparing profile data . . .

```

Figure 4 - Example of usage when previous choices are remembered

Finally, the model is run according to the user's choices and returns an output (in this case 4 sequences were analysed):

```

Would you like the program to remember your choices?
(y/n)

>>>> y

Your choices have been registered!
The next time the program is ran, you'll be asked if you want
to execute your previous choices.

=====

Building HMM profiles: [#####] 100.00%
Scoring sequences:    [#####] 100.00%

Output data saved successfully to:
C:\Users\Asus\Documents\GitHub\HIV_HiddenMarkovModels_naive-treated_Prediction\output_data\output.csv

>>B.JP.2009.NMC127_clone_07.AB731667 classified as B_treated

>>B.JP.2009.NMC127_clone_27.AB731668 classified as B_treated

>>B.JP.2011.NMC851C_clone_13.AB731669 classified as B_treated

>>B.JP.2011.NMC851C_clone_48.AB731670 classified as B_treated

```

Along the way of this UI, there are multiple “fail-safe” checkpoints that avoid breaks in the program, so that the user doesn't lose his progress. Here are some examples:

```

=====
Hello Dr. Asus! Ready to find who's naive and who's treated?
=====

Please write the HIV subtype of the sample(s).
("A", "B", "02_AG" or "unknown")

>>>> not a valid subtype

Invalid subtype!

Please write the HIV subtype of the sample(s).
("A", "B", "02_AG" or "unknown")

>>>> b

=====

```

Figure 6 – Invalid subtype error

```

Specify Protease Inhibitor (PI) codons.
List of codon positions must be given in the format 'pos1, pos2, ..., pos N'.

PI codons >>>> not a list of integers

Wrong format! Try again...

Specify Protease Inhibitor (PI) codons.
List of codon positions must be given in the format 'pos1, pos2, ..., pos N'.

PI codons >>>> 1;2;3

Wrong format! Try again...

Specify Protease Inhibitor (PI) codons.
List of codon positions must be given in the format 'pos1, pos2, ..., pos N'.

PI codons >>>> 1,2,3

Specify Protease Inhibitor (RT) codons.
List of codon positions must be given in the format 'pos1, pos2, ..., pos N'.

RT codons >>>> .

```

Figure 8 - Wrong format error when specifying list of codons

Test command

This command's purpose is to provide the user with performance metrics for a model trained and tested based on the user's input (this command is more useful for developers).

To execute the *test* command the user must type “-t” or “—test”, the first part of this command's UI is very similar to the *classify* command's:

```
C:\Users\Asus\Documents\GitHub\HIV_HiddenMarkovModels_naive-treated_Prediction\code>python hiv_hmm_pred.py -t

=====

This section trains and then tests the model based is your
input and returns performance info (for each iteration and for
overall performance).
=====

Please write which HIV subtype you intend to model.
("A", "B", "02_AG" or "unknown")

>>> unknown

I see that you want to model unknown subtypes.

Would you like the model to predict subtype as well?
(y/n)

>>> y

=====

Do you want the model to discard the major DRM codons?
(y/n)

>>> y

The major DRM codons considered by the model are the following:
» PI codons: [30, 32, 33, 46, 47, 48, 50, 54, 76, 82, 84, 88, 90]
» RT codons: [184, 65, 70, 74, 115, 41, 67, 70, 210, 215, 219, 100, 101, 103, 106, 138, 181, 188, 190, 230]

Do you want to specify a different list of DRM codons?
(y/n)

>>> n

=====
```

The user is asked: what's the subtype of the sequences (and if he doesn't know it if he wants to predict it), if he wants to remove major DRM codons (and if he wants to specify them).

After this, there's new parameters to input, first is the test fraction the user pretends to utilize (and the respective error messages):

```
=====

Which fraction of the data do you want to use for testing?
The answer must be a number between 0 and 1 and the decimal
separator must be a dot '.'

>>> 0

The answer must be a number between 0 and 1! Try again...

Which fraction of the data do you want to use for testing?
The answer must be a number between 0 and 1 and the decimal
separator must be a dot '.'

>>> 1

The answer must be a number between 0 and 1! Try again...

Which fraction of the data do you want to use for testing?
The answer must be a number between 0 and 1 and the decimal
separator must be a dot '.'

>>> 0,2

Wrong format! Try again...

Which fraction of the data do you want to use for testing?
The answer must be a number between 0 and 1 and the decimal
separator must be a dot '.'

>>> 0.2

=====
```

Next, the user is asked if he wants to downsize the data used in training/testing the model:

```
=====
Do you want to use a downsized version of the dataset?
(y/n)

>>>> y

Indicate de maximum number of samples for training:
(if you don't want to limit training input 'none')

>>>> None

Indicate de maximum number of samples for testing:
(if you don't want to limit testing input 'none')

>>>> 2

=====
```

Then the user is asked if he wants to balance the dataset (same number of treated and naïve sequences) via under sampling:

```
=====
Do you want to use a balanced dataset?
(same number of naive and treated sequences but, consequently
less data)

>>>> y

=====
```

In the end, the program asks the user how many iterations of the desired model should be ran:

```
=====
Indicate the number of iterations:
>>>> 2
=====
```

Finally, the program trains and tests the model based on the user's inputs. The performance metrics are *printed* for each iteration and, in the end the average metrics are *printed* as well. The model's output data during the whole process and from the last iteration are saved in the "output_data" directory:

```

»»»»»»»»»»»»»»»»»»»»»»»»»»»»»»»»»» ITERATION 1/2 »»»»»»»»»»»»»»»»»»»»»»
Building HMM profiles: [#####] 100.00%
Scoring sequences:    [#####] 100.00%


Accuracy: 50.0%
      precision recall f1-score
B_naive       0.00     0.0   0.000000
B_treated     0.50     1.0   0.666667
mean          0.25     0.5   0.333333


»»»»»»»»»»»»»»»»»»»»»»»»»»»»»»»»»» ITERATION 2/2 »»»»»»»»»»»»»»»»»»»»»»

Building HMM profiles: [#####] 100.00%
Scoring sequences:    [#####] 100.00%


Accuracy: 75.0%
      precision recall f1-score
B_naive       1.000000   0.50   0.666667
B_treated     0.666667   1.00   0.800000
mean          0.833333   0.75   0.733333


»»»»»»»»»»»»»»»»»»»»»»»»»»»»»»»»»» FINAL RESULTS »»»»»»»»»»»»»»»»»»»»»»

Accuracy: 62.5%
      precision recall f1-score
B_naive       1.000000   0.250   0.400000
B_treated     0.571429   1.000   0.727273
mean          0.785714   0.625   0.563636

```

Discussion

The findings of this study demonstrate the feasibility of Hidden Markov Models (HMMs) in classifying HIV-1 *pol* gene sequences based on antiretroviral treatment (ART) history. With classification accuracies of 67.77% for subtype B and 65.97% for CRF02_AG, the model exhibited moderate effectiveness, reinforcing the potential of probabilistic sequence-based classification for identifying ART-naïve and ART-experienced patients. However, the observed discrepancies in precision and recall, particularly for CRF02_AG, highlight inherent challenges in accurately distinguishing between treatment groups.

One major limitation stems from the presence of transmitted drug resistance mutations (TDRMs), which may introduce classification biases. Although major resistance codons were removed to mitigate this issue, the persistence of residual confounding effects suggests that additional sequence features beyond known resistance-associated sites may influence classification. The dataset imbalance, where ART-naïve sequences outnumbered ART-experienced ones, necessitated undersampling, which may have impacted model generalizability. Future work could explore alternative data balancing techniques, such as synthetic oversampling of minority-class sequences, to enhance performance.

Looking ahead, future research could focus on developing a confidence measure for HMM predictions to quantify the reliability of classifications, aiding clinical decision-making. Additionally, exploring other regions of the HIV-1 genome, such as *gag* or *env*, may provide complementary sequence features that improve classification accuracy and offer deeper insights into treatment-related genomic patterns. Finally, investigating the performance of alternative machine learning models, such as Random Forests, could provide comparative insights and potentially enhance predictive accuracy.

Overall, this study highlights the potential of HMM-driven classification as a valuable aid in HIV-1 treatment stratification. While refinements are needed to improve accuracy and generalizability, this method provides a computationally efficient and biologically interpretable approach to addressing the challenge of uncertain treatment histories in HIV-1 patients. Future efforts to incorporate confidence measures, explore additional genomic regions, and test alternative models will further enhance the utility and reliability of sequence-based classification in clinical applications.

Bibliography

- D. Durand. (2019). Hidden Markov Models. In *Computational Biology*.
- Degirmenci, A. (2014). *Introduction to Hidden Markov Models*.
- Pimentel, V., Pingarilho, M., Sebastião, C. S., Miranda, M., Gonçalves, F., Cabanas, J., Costa, I., Diogo, I., Fernandes, S., Costa, O., Corte-Real, R., Martins, M. R. O., Seabra, S. G., Abecasis, A. B., & Gomes, P. (2024). Applying Next-Generation Sequencing to Track HIV-1 Drug Resistance Mutations Circulating in Portugal. *Viruses*, 16(4).
<https://doi.org/10.3390/v16040622>
- Portugal. Ministério da Saúde. Direção-Geral da Saúde/Instituto Nacional de Saúde Doutor Ricardo Jorge. (2024). *Infeção por VIH em Portugal - 2024*.